KERNEL INTEGRITY ENFORCEMENT WITH HLAT IN A VIRTUAL MACHINE

Gao Chao <chao.gao@intel.com>

October 29, 2020

Agenda

Part1: Address Translation Integrity

Part2: HLAT Introduction

Part3: Kernel Integrity Enforcement With HLAT

PART1: ADDRESS TRANSLATION INTEGRITY

Access Control via Page Table

Rich permission bits (e.g. Present, R/W, U/S, XD)

Access Control

- Read-only Kernel Text/rodata
- Writable XoR executable
- Kernel/User memory segregation

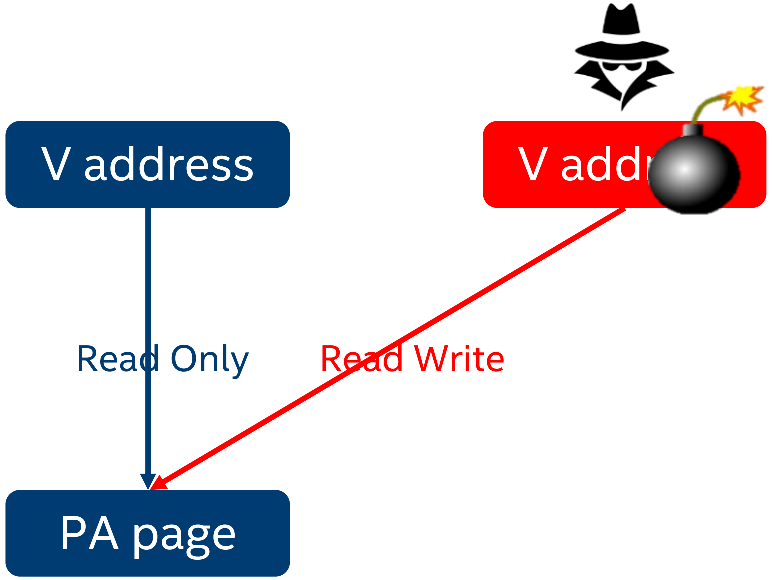
Page Table/Translation Integrity is Important!

Get page table location from task_struct → Overwrite page table to set kernel code writable → Overwrite the beginning of a syscall with shellcode

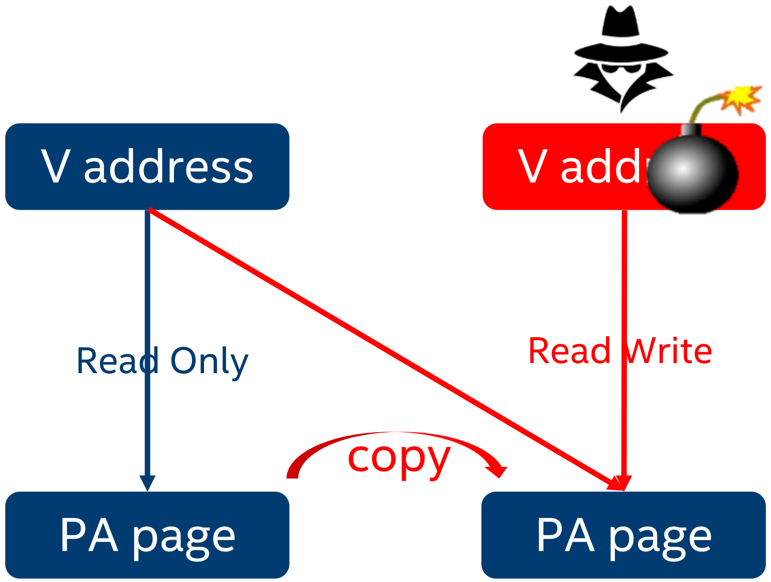
https://www.ndss-symposium.org/wp-content/uploads/2017/09/ndss2017_05B-4_Davi_paper.pdf

Typical Page Table Override Attacks

Alias Mapping

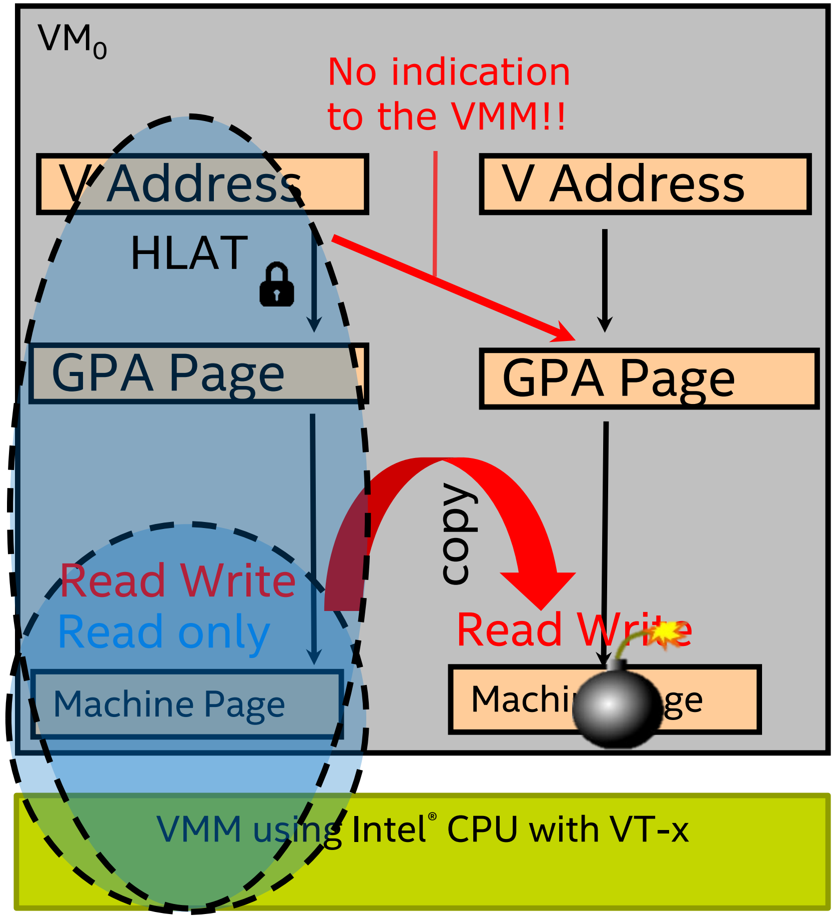


Page Remapping



EPT Can Help but Limited

- EPT can only enforce access control to machine page.
- Page remapping attacks cannot be detected if VMM doesn't monitor guest page table changes.
- Monitor guest page table changes -> high performance penalty
  - Trap "MOV to CR3"
  - Write-protect CR3 page table
  - audit each attempt to change page table



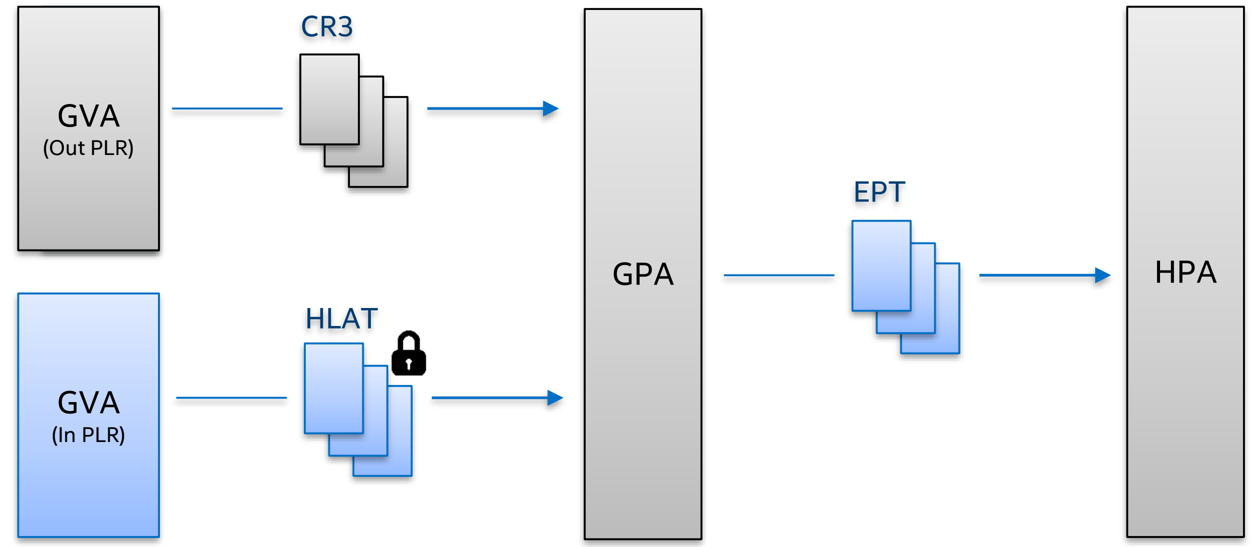
PART2: HLAT INTRODUCTION

Hypervisor-managed Linear Address Translation (HLAT)

SPEC: https://software.intel.com/content/www/us/en/develop/download/intel-architecture-instruction-set-extensions-programming-reference.html

Goal: enforce (guest) translation integrity with VMM

Key Idea

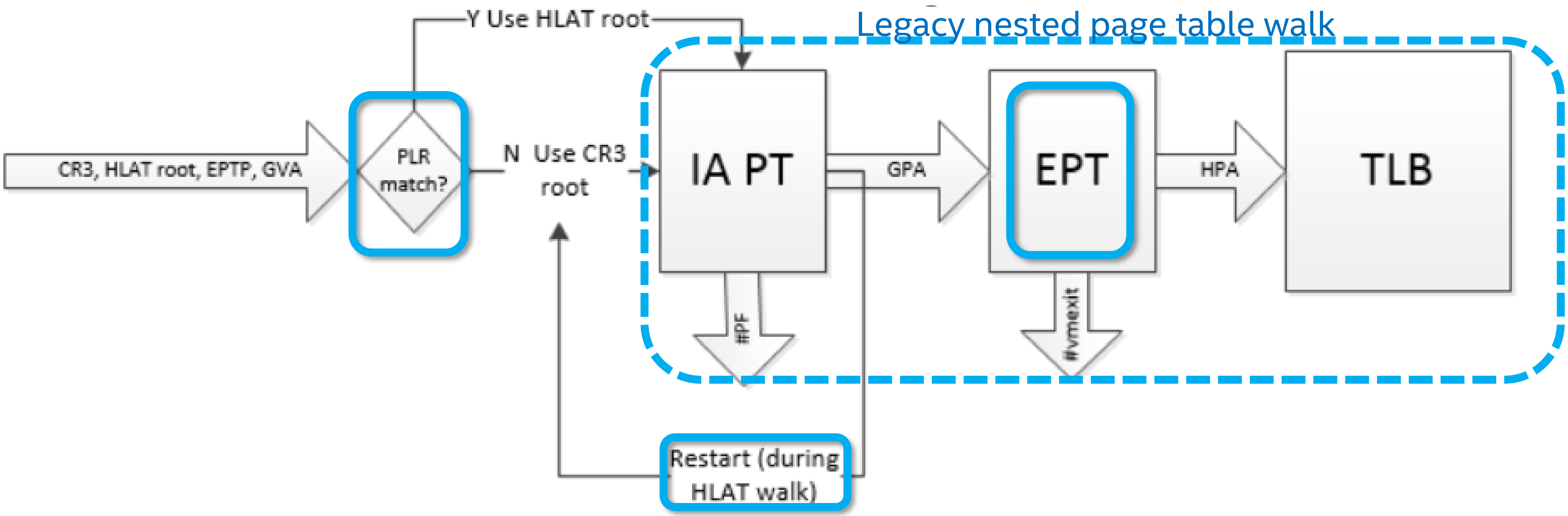


Benefits

Security: invulnerable to page remapping attack

Efficiency: no interception to CR3 page table modification (compared with EPT-based page table protection).

HLAT – Nested Page Table Walk



HLAT – Page Structures

- Same as IA32e page structure
- Support 5-level and 4-level
- Bit 11: “Restart”

XD[3]	Prot. Key[4]	Ignored	Rsvd.	Address of 4KB page frame	R	gn.	G	PAT	D	A	PCD	PWT	U/S	R/W	1	PTE: 4KB page

HLAT Terminal Fault

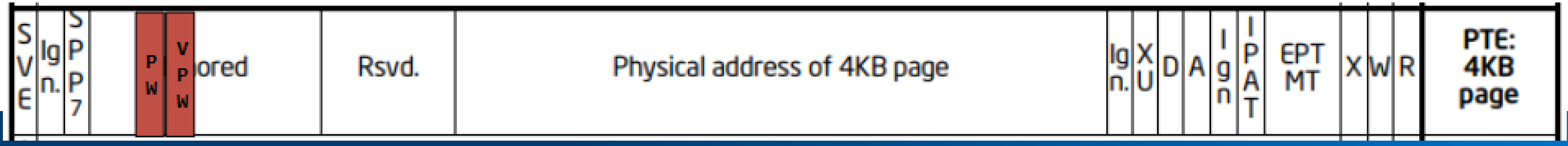
- non-present or reserved bits
- new page fault error code: bit 7 – “HLAT Terminal Fault”

EPT Control Bit “Paging-Write”

- **“Paging Write”** allows CPU to update A/D bits on pages that are non-writable to software.

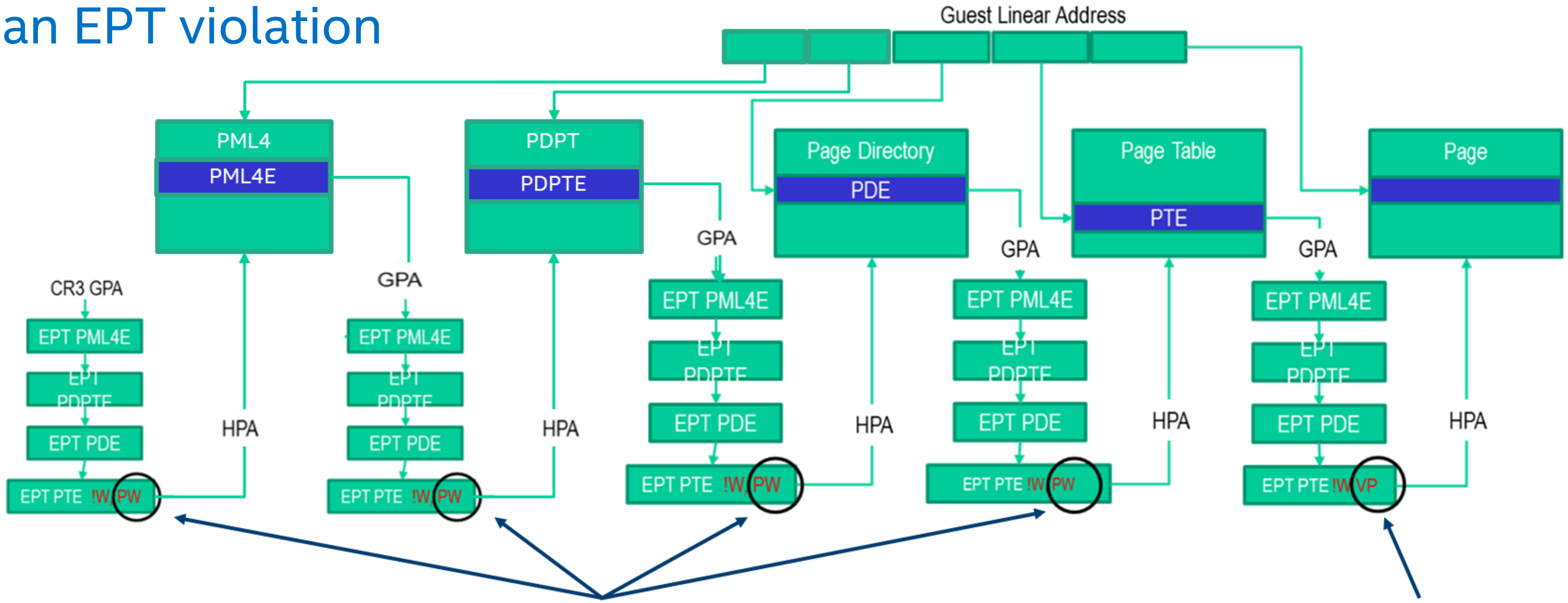
EPT Control Bits	w	!w	!w|pw
Software writes	Allowed	Denied	Denied
A/D bits update	Allowed	Denied	Allowed

- Improve efficiency of read-only guest page table (under EPT):
  - Reduce VM-exits due to A/D bits update
  - Relieve VMM from A/D bits emulation



EPT Control Bit “Verify Paging-write”

- **“Verify Paging-write”** enforces that all leaf guest-paging-structure pages encountered during the nested walk have PW set (under EPT), else generates an EPT violation

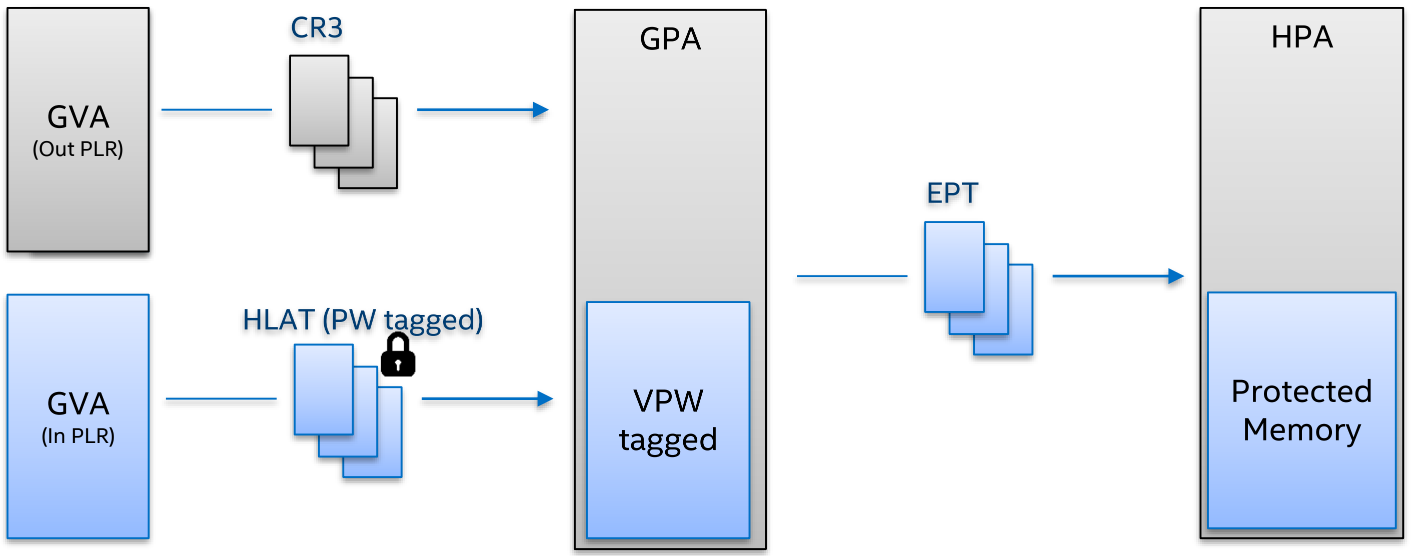


Enables VMM to complete CPU A/D bit updates on access-controlled Guest paging structure pages without EPT violation VM exits

Verifies that page-walk occurred through VMM access-controlled pages only

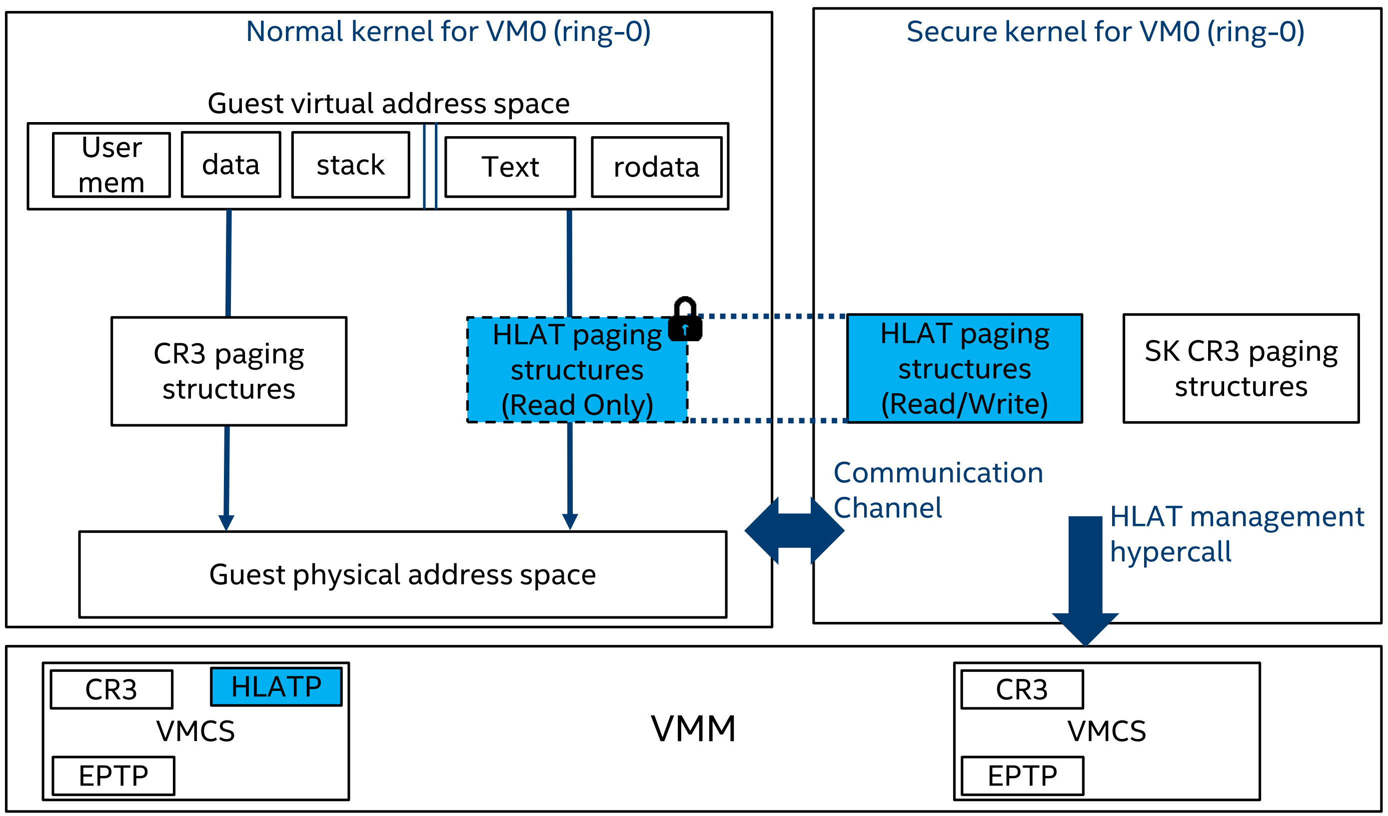
Prevent Alias Mapping with PW & VPW

VPW tagged guest memory can only be accessed with HLAT (PW tagged page table)



PART3: KERNEL INTEGRITY ENFORCEMENT WITH HLAT

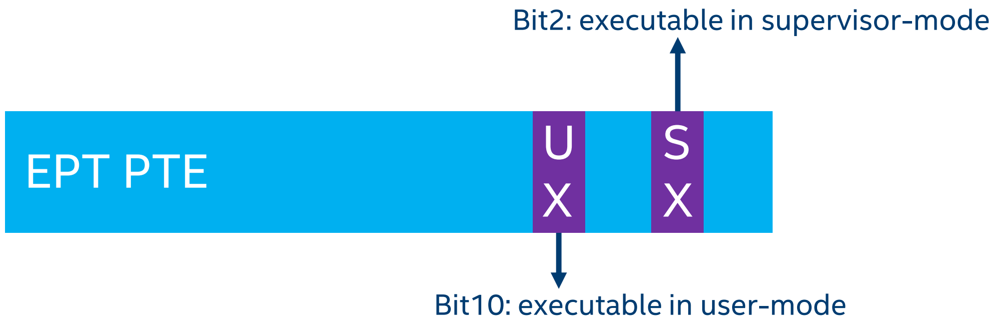
Protect Translation for Kernel Text/rodata



- A trusted entity is required to manage HLAT paging structures. It can be the secure kernel or the VMM.
- Tag text/rodata with VPW flag under EPT
- Tag HLAT with PW flag under EPT

Set up/Tear down Protected Translations

- Usages
  - Load/unload modules
  - OS reboot



- Teardown protected translations may be abused
  - Mitigation: mode-based execution control for EPT (for code only)
  - Revoke guest memory's executable permission in supervisor-mode on un-protection.

Text Patching

- Valid kernel code changes at runtime
  - Usages: function tracing, jump label, alternatives, etc.

- Need an approval policy to avoid being abused by attackers
  - Based on the location to patch, what the patch does

Security Values

- Defend against code injection attacks (e.g. hook syscall) and raise the bar of exploiting kernel vulnerability.

- Even if kernel is compromised, VMM can ensure the integrity of guest kernel’s code and rodata.

Summary

- Restricting accesses to kernel code/rodata can reduce attack surface of kernel.
- Page table attacks can bypass such restriction.
- HLAT provides an efficient way to enforce the integrity of guest translation.
- VMM can use HLAT and EPT control bits to enforce kernel code/rodata integrity.

THANKS! Q&A